Race conditions: database invariants and idempotent reservations

Researched 2026-11-07.

Strengths and best use

The last-ticket example makes a concurrency failure concrete. Two requests can both observe available stock before either writes the reservation. The useful lesson is to enforce the invariant in the database operation and then test competing requests. PostgreSQL documents row locks, isolation levels, and constraints for this purpose. Locks (https://www.postgresql.org/docs/current/explicit-locking.html) - Isolation (https://www.postgresql.org/docs/current/transaction-iso.html)

A small reference pattern

In a disposable PostgreSQL fixture, create one inventory row and use a conditional update. A successful request receives a returned row. A request receiving no row must handle unavailable stock. The transaction and its result must drive the application response.

```
CREATE TABLE ticket_stock (
  event_id integer PRIMARY KEY,
  remaining integer NOT NULL CHECK (remaining >= 0)
);
INSERT INTO ticket_stock VALUES (42, 1);
BEGIN;
UPDATE ticket_stock
SET remaining = remaining - 1
WHERE event_id = 42 AND remaining > 0
RETURNING event_id, remaining;
COMMIT;
```

Verify the whole workflow

Start concurrent clients at a barrier and assert exactly one successful reservation. Then test retries, cancellation, transaction rollback, and a timeout after the database commits. A unique idempotency record can make repeated client requests return the original result. A payment provider's timeout also needs reconciliation with its actual payment state. Serializable transactions can raise retryable failures, so implement bounded retries around the full transaction. Constraints (https://www.postgresql.org/docs/current/ddl-constraints.html)

Reusable prompt

```
Find the invariant in this reservation workflow. Design a database-enforced write, idempotent request handling, and a concurrent-client test. Cover rollback and an uncertain payment outcome. State the actual isolation level and which failures the application retries.
```

Project idea

Build a last-item reservation fixture with a barrier, result ledger, and retry injection. Report successful reservations and final stock together.

Original sources

- https://www.instagram.com/reel/Det6xAWgLk4/